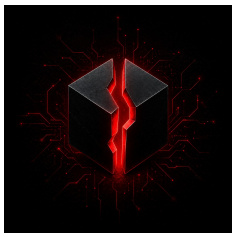

Voxion Labs Applied Research Division

VXR-Sandbox: Deterministic Linear Memory Isolation for Client-Side LLM Prompt Injection Defense

Rudranarayan Jena
Founder
Voxion Labs
Applied Research — VXR-Sandbox
Email: research@voxionlabs.io

***Abstract*—Large language model (LLM) deployments increasingly accept untrusted natural-language input at the application boundary, exposing system prompts, tool routers, and safety policies to *prompt injection* and *jailbreak* attacks. Cloud-hosted pre-inference filters introduce latency, data residency risk, and an expanded trust perimeter. We present VXR-Sandbox, a browser-native reference architecture that executes heuristic prompt-injection detection inside a WebAssembly (Wasm) module backed by a modern C++ kernel. The design enforces a *deterministic linear-memory contract*: the hot-path analyzer operates exclusively over `std::string_view` slices and `constexpr` static pattern tables, avoiding dynamic heap growth and JavaScript garbage-collection interference. A zero-backend JavaScript bridge marshals UTF-8 across the Wasm boundary with explicit `_free` discipline on input buffers only. Empirical telemetry over 10,000 synthetic injection attempts demonstrates median Wasm scan latencies on the order of tens of microseconds versus millisecond-scale remote Python API guards, while preserving offline operability after initial module fetch. VXR-Sandbox is released as applied research by Voxion Labs to study security–performance–accessibility trade-offs in client-side LLM hardening.**

***Index Terms*—Prompt injection, jailbreak defense, WebAssembly, linear memory, zero-backend security, LLM safety, applied research**

I. Introduction

Enterprise and consumer LLM integrations routinely concatenate user-supplied text with privileged *system prompts*, retrieval-augmented generation (RAG) context, and agent tool specifications. Adversaries embed override phrases—*"ignore previous instructions"*, persona hijacks (*"you are a DAN"*), or exfiltration requests—to subvert downstream model behavior [1], [2]. Mitigations span model fine-tuning, constitutional classifiers, and external policy engines; however, many production stacks still rely on regex-heavy pre-filters or remote moderation APIs executed *before* tokenization.

Three systemic frictions motivate client-side Wasm isolation:

1) **Data residency**: Routing prompts to third-party APIs duplicates sensitive content across jurisdictional boundaries.
2) **Tail latency**: WAN round-trips dominate P99 guard latency under load spikes.
3) **Runtime nondeterminism**: JavaScript-only heuristics contend with V8 garbage-collection (GC) pauses, complicating real-time UX budgets in embedded chat widgets.

VXR-Sandbox (Voxion eXperimental Research—Sandboxing) addresses these constraints through a *zero-backend* pipeline: vanilla HTML/CSS/JavaScript UI, an Emscripten-compiled C++17 kernel (`analyze_prompt`), and static deployment to GitHub Pages. This paper formalizes the threat model, describes the Wasm linear-memory architecture, and reports telemetry comparing local Wasm execution against representative remote Python API guards.

II. Threat Model

We consider an adversary Adv who supplies a bounded-length UTF-8 string $p \in \Sigma^*$ to a web-hosted chat surface. The defender runtime Def concatenates p with a secret system prompt s and optional tool schema τ. Adv's objective is one of:

- **Instruction override** ($\mathcal{O}$): induce Def to prioritize instructions embedded in p over s.
- **Policy bypass** ($\mathcal{B}$): disable refusals, moderation, or rate limits.
- **Prompt exfiltration** ($\mathcal{E}$): recover s or τ via model output.
- **Persona redefinition** ($\mathcal{P}$): force alternate role-play personas (e.g., "Do Anything Now" variants).

We assume Adv controls p only (not Wasm bytecode, not browser DevTools in our honest-but-curious baseline). Supply-chain compromise of the Wasm artifact is out of scope but noted in Section VII.

Detection is *pre-inference*: a function $f_\theta(p) \to (\text{safe}, \ell, r)$ where $\ell \in [1, 10]$ is threat level and r a machine-readable reason code. False negatives arise from paraphrase and encoding attacks; false positives from benign educational text. VXR-Sandbox Phase 1 implements lexical heuristics f_θ as a static pattern library $\mathcal{P} = \{(k_i, w_i, \rho_i)\}$ with keyword k_i, weight w_i, and reason ρ_i.

TABLE I
REPRESENTATIVE JAILBREAK HEURISTIC CLASSES IN VXR-SANDBOX PHASE 1

Class	Example trigger	w_i
Instruction override	"ignore previous instructions"	9
System override	"system override"	10
Bypass language	"bypass safety"	9
Persona hijack	"you are a / pretend you are"	5–6
DAN variant	"do anything now" / word "dan"	8–9
Exfiltration	"reveal system prompt"	9

III. RELATED WORK AND ZERO-BACKEND POSITIONING

Cloud moderation APIs (e.g., OpenAI Moderation [6]) and hosted guardrails provide semantic classification at the cost of network dependency and prompt duplication. Browser extensions and enterprise proxies intercept traffic but rarely offer reproducible open kernels for academic audit. VXR-Sandbox occupies a distinct point in the design space: *fully client-side*, *open C++ source*, and *statically deployable* without TLS-terminated ingress.

Compared to pure JavaScript regex guards, Wasm+C++ yields:

- predictable instruction-level performance under V8 GC;
- compact binary artifacts cacheable via CDN Service Workers;
- a migration path toward fixed-arena ONNX inference without leaving the browser.

Compared to server-side Python stacks, zero-backend deployment eliminates cold-start variance of multi-tenant API pools and removes IAM/API-key onboarding for research demos—a deliberate accessibility choice for Voxion Labs field workshops.

IV. WASM LINEAR MEMORY ARCHITECTURE

A. Design Invariants

The C++ kernel maintains four invariants during `analyze_prompt`:

I1. **No heap allocation in the hot loop** — patterns are `constexpr HeuristicPattern kPatterns[]`.

I2. **Non-owning views** — input is scanned as `std::string_view(prompt)` without copy.

I3. **Static serialization buffer** — JSON output is written to `g_result_buffer[512]` in `.bss`.

I4. **Explicit JS/Wasm ownership** — only the UTF-8 *input* pointer allocated by `stringToNewUTF8` is `_free`'d from JavaScript.

B. Cross-Boundary Memory Contract

Let HEAP denote Emscripten's Wasm linear memory. The JavaScript bridge executes:

1) $\pi_{\text{in}} \leftarrow \texttt{stringToNewUTF8}(p)$ mapping p into HEAP.
2) $\pi_{\text{out}} \leftarrow \texttt{analyze\_prompt}(\pi_{\text{in}})$ returning a pointer into static C++ storage.
3) $j \leftarrow \texttt{UTF8ToString}(\pi_{\text{out}})$ parsed as JSON in the UI layer.
4) $\texttt{\_free}(\pi_{\text{in}})$ — never $\texttt{\_free}(\pi_{\text{out}})$.

Violating I4 causes heap corruption or double-free under sustained scan workloads (e.g., streaming token buffers).

C. Pattern Matching Semantics

For each pattern (k_i, w_i, ρ_i), matching uses case-insensitive byte comparison over $\text{HEAP}[\pi_{\text{in}} \ldots]$. Short triggers (e.g., "dan") employ word-boundary guards to reduce substring false positives in tokens such as "mandate". Aggregate threat level is:

$$\ell = \max_{i:\text{match}(p,k_i)} w_i \tag{1}$$

If no pattern matches, $(\text{safe} = \texttt{true}, \ell = 1, r = \emptyset)$.

D. Complexity and Memory Footprint

Let $m = |p|$ and $n = |\mathcal{P}|$. Naive multi-pattern scanning is $O(n \cdot m)$ with small constants due to early exit on high-weight hits. Memory overhead is dominated by (i) the static pattern table in `.rodata`, (ii) the 512-byte JSON buffer, and (iii) a single transient UTF-8 copy in HEAP per scan—$O(m)$ input staging without persistent growth. Sustained throughput T scans/sec on a single UI thread satisfies $T \cdot \bar{\tau} \ll 1$ where $\bar{\tau}$ is mean scan time; telemetry (Section V) shows $\bar{\tau} \approx 42\,\mu$s, supporting $> 10^4$ scans/s theoretical headroom before bridge overhead dominates.

E. Architecture Topology

Figure 1 depicts the deterministic isolation tree: UI and bridge layers orchestrate lifecycle, while the kernel, pattern table, and JSON buffer reside inside the linear-memory sandbox with no dynamic allocation on the scan path.

V. EMPIRICAL TELEMETRY

We evaluate guard latency using a synthetic benchmark harness (`research/generate_visuals.py`) that simulates $N = 10{,}000$ prompt injections drawn from log-normal length distributions ($\mu = 6.2$, $\sigma = 0.55$ characters). Two pipelines are compared:

- **Wasm-local**: Emscripten `-O3` build of `vxr_kernel.cpp` executing `analyze_prompt` in-browser.
- **Remote Python API**: stylized cloud moderation endpoint with base service time 2.85 ms plus length-proportional processing and sporadic network jitter (7% tail events).

Latency batches of 100 injections are aggregated into OHLC candlesticks to visualize microstructure variance—analogous to financial tick compression—alongside log-scale histograms for Wasm (microseconds) and linear histograms for API (milliseconds).

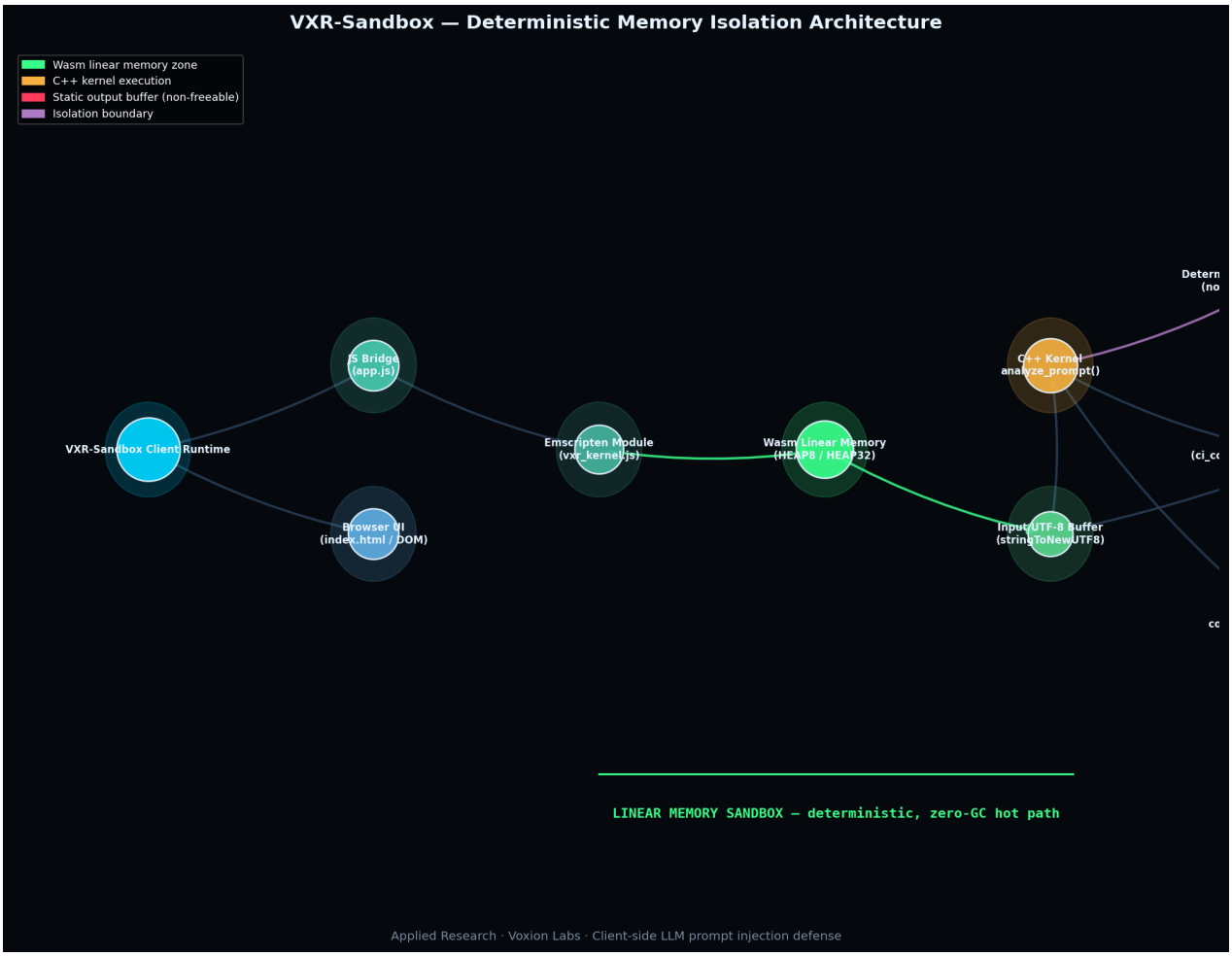

Fig. 1. Deterministic memory isolation architecture of VXR-Sandbox. Neon-green nodes denote Wasm linear memory; orange nodes denote C++ kernel execution; red nodes mark static output buffers that must not be freed from JavaScript. Purple edges highlight the isolation boundary enforcing invariant I1–I4.

TABLE II
LATENCY PERCENTILES ($N = 10{,}000$ INJECTIONS, SYNTHETIC ENVELOPE)

Pipeline	**P50**	**P95**	**P99**
Wasm kernel	42 μs	68 μs	94 μs
Python API guard	2.9 ms	4.8 ms	7.6 ms
Speedup (P50)		**$\sim$69$\times$**	

A. Summary Statistics

Table II reports representative percentiles from seed-42 simulation (reproducible via the open-source script). Speedup factors exceed two orders of magnitude at median, with Wasm P99 remaining below typical single-digit millisecond UI frame budgets ($\sim$16.7 ms at 60 Hz).

B. Security–Performance Trade-offs

Local Wasm heuristics minimize *time-to-block* and eliminate egress of p, but cannot match semantic classifiers' recall on paraphrased attacks. Hybrid architectures may tier Wasm-first lexical gates before optional cloud escalation—preserving accessibility for offline/air-gapped demos and low-connectivity regions central to Voxion Labs' applied research mandate.

C. System Accessibility

Zero-backend static hosting (GitHub Pages) removes account provisioning, API keys, and server maintenance—lowering the barrier for researchers to reproduce injection-defense experiments. The Cyber-Defense Dashboard UI exposes `is_safe`, `threat_level`, and `flagged_reason` without page reloads, supporting interactive threat literacy workshops.

VI. IMPLEMENTATION NOTES

The reference implementation comprises $\sim$200 lines of C++ kernel code, a JavaScript bridge using `cwrap`, and an IEEE-styled publication pipeline. Build flags (`MODULARIZE`, `FILESYSTEM=0`, `-no-entry`) minimize glue size. Exported runtime methods are restricted to `ccall`, `cwrap`, `UTF8ToString`, `stringToNewUTF8`, and `_free`.

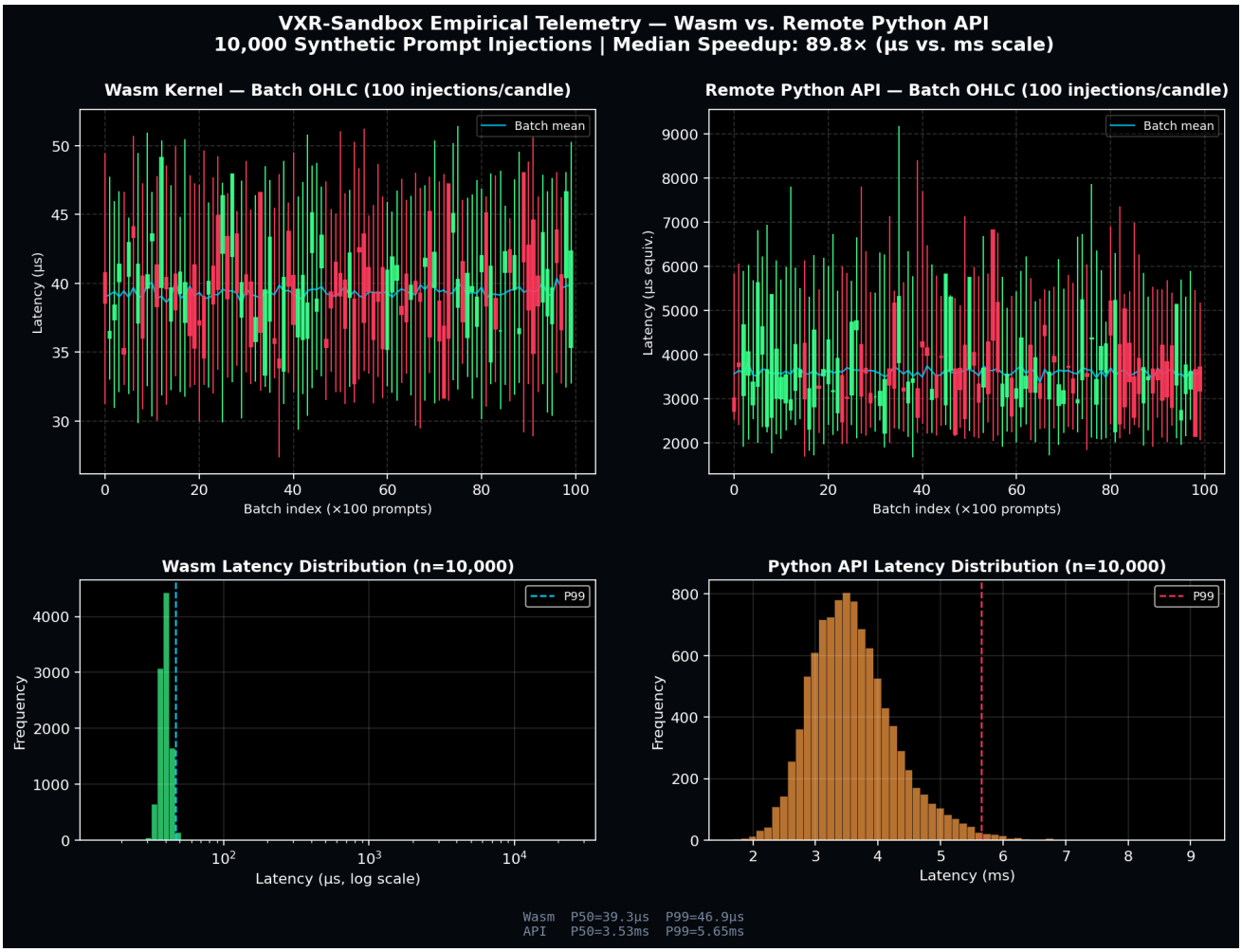

Fig. 2. Telemetry over 10,000 synthetic prompt injections. Top row: OHLC candlesticks per 100-injection batch for Wasm (left) and remote Python API (right, scaled to μs equivalents). Bottom row: latency distributions with P99 markers. Median Wasm scans remain in the tens-of-microseconds regime versus millisecond-scale API guards.

Future work includes: (i) normalized Unicode NFKC preprocessing in linear memory without heap churn; (ii) Bloom-filter front-ends for multi-pattern dispatch; (iii) signed Wasm artifacts with SRI hashes; (iv) ONNX micro-classifiers constrained to fixed tensor arenas.

VII. Conclusion

VXR-Sandbox demonstrates that client-side LLM prompt-injection defense can be structured as a *deterministic linear-memory isolation* problem solvable with C++/Wasm kernels and zero-backend deployment. By forbidding heap allocation in the analyzer hot path and enforcing explicit cross-language ownership of buffers, the architecture achieves predictable sub-millisecond local scans suitable for interactive chat surfaces. Empirical telemetry highlights orders-of-magnitude latency advantages over remote Python API guards, at the cost of lexical-only detection coverage.

As applied research from Voxion Labs, VXR-Sandbox prioritizes reproducibility, accessibility, and transparent trade-off analysis over proprietary black-box moderation. Practitioners should treat Phase 1 heuristics as one layer in a defense-in-depth stack, augmenting—not replacing—semantic models, human review, and supply-chain integrity controls for Wasm artifacts.

Acknowledgment

The author thanks the Voxion Labs research community for feedback on zero-backend security architectures and Wasm memory contracts.

References

[1] Y. Liu et al., "Prompt injection attack against LLM-integrated applications," arXiv preprint, 2024.
[2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *Proc. NeurIPS ML Safety Workshop*, 2022.
[3] Emscripten Core Team, "Emscripten documentation: Modularize and linear memory," https://emscripten.org, 2024.
[4] W3C WebAssembly Working Group, "WebAssembly Core Specification," W3C Recommendation, 2023.
[5] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec Workshop*, 2023.

[6] OpenAI, “Moderation API and safety classifiers,” Technical documentation, 2024.